Object Oriented Programming
Assignment 4

Instructor: Mam Hina Iqbal
TA: Taimoor Shakeel (l226822@lhr.nu.edu.pk)
Section: BCS-2F
Due Date: Tbd

Question: Smart Ride Booking System

Marks: 30 | Concepts: Polymorphism + Exception Handling

Problem Overview

You are required to build a console-based Ride Booking System in C++ that demonstrates deep runtime polymorphism through multiple ride types with different pricing strategies, validation rules, and behaviors. Each ride type calculates fare differently and applies unique constraints. The system must dynamically decide behavior at runtime using base class pointers. Additionally, the system must handle runtime errors using try-catch blocks.

Concept Mapping

Runtime Polymorphism → Base class Ride with multiple virtual functions
Function Overriding → Each ride type overrides fare, validation, display
Abstract Class → Ride contains pure virtual methods
Dynamic Binding → Base pointer calls correct derived implementation
Exception Handling → Invalid rides, surge errors, negative distance

Class 1: Ride (Abstract Base Class)

```
class Ride {
protected:
    char* riderName;
    double distance;
public:
    Ride(const char* name, double dist);

    virtual double calculateFare() = 0;
    virtual void applySurgePricing() = 0;
    virtual void validateRide() = 0;
    virtual void displayRideDetails() = 0;

    virtual ~Ride();
};
```

Class 2: EconomyRide

```
class EconomyRide : public Ride {
public:
    EconomyRide(const char* name, double dist);

    double calculateFare() override;
    void applySurgePricing() override;
    void validateRide() override;
    void displayRideDetails() override;
};
```

Logic: Base fare = 50 + (distance × 20), Surge = +10%, Throw exception if distance ≤ 0

Class 3: PremiumRide

```
class PremiumRide : public Ride {
private:
    double luxuryFee;
public:
    PremiumRide(const char* name, double dist, double fee);

    double calculateFare() override;
    void applySurgePricing() override;
    void validateRide() override;
    void displayRideDetails() override;
};
```

Logic: Base fare = 100 + (distance × 40) + luxuryFee, Surge = +25%, Throw exception if luxuryFee < 0

Class 4: BikeRide

```
class BikeRide : public Ride {
public:
    BikeRide(const char* name, double dist);

    double calculateFare() override;
    void applySurgePricing() override;
    void validateRide() override;
    void displayRideDetails() override;
};
```

Logic: Base fare = distance × 10, No surge allowed, Throw exception if distance > 50

Class 5: SharedRide

```
class SharedRide : public Ride {
private:
    int passengers;
public:
    SharedRide(const char* name, double dist, int p);
```

```
    double calculateFare() override;
    void applySurgePricing() override;
    void validateRide() override;
    void displayRideDetails() override;
};
```

Logic: Total fare = distance × 25, Fare split = total / passengers, Throw exception if passengers < 1

Class 6: RideManager

```
class RideManager {
private:
    Ride** rides;
    int count;
public:
    RideManager();
    ~RideManager();

    void addRide(Ride* r); void processRides(); //iterates over all rides using a loop and, through base class pointers, calls validateRide(), applySurgePricing(), calculateFare(), and displayRideDetails(). This entire processing must be implemented using try-catch blocks so that if one ride fails, the remaining rides continue execution
    void displayAllRides();
};
```

Polymorphism Requirements:

Store all ride types in Ride* array, Call calculateFare(), applySurgePricing(), validateRide(), displayRideDetails() using base pointer, Ensure correct method is called at runtime

Exception Handling Requirements:

```
try {
    ride->validateRide();
    ride->applySurgePricing();
    double fare = ride->calculateFare();
}
catch (const char* msg) {
    cout << "Error: " << msg << endl;
}
```

What main() Must Demonstrate

```
int main() {
   // Step 1: Create Ride Objects (Independently)
   Ride* r1 = new EconomyRide("Ali", 5);
   Ride* r2 = new PremiumRide("Sara", 10, 200);
   Ride* r3 = new BikeRide("Ahmed", 60);      // invalid case
   Ride* r4 = new SharedRide("Zain", 20, 4);
   Ride* r5 = new SharedRide("Hassan", 15, 0); // invalid case

   // Step 2: Create Ride Manager
   RideManager manager;

   // Step 3: Add Rides to Manager
   manager.addRide(r1);
   manager.addRide(r2);
   manager.addRide(r3);
   manager.addRide(r4);
   manager.addRide(r5);

   // Step 4: Process All Rides
   cout << "Processing All Rides...\n\n";
   manager.processRides();
```